LogHawk 高性能分布式日志采集与处理系统

1. 核心能力

- 实时采集
  - 监控日志文件，低延迟读取新增内容，支持多文件、日志轮转
- 高并发处理
  - 多进程并行解析、过滤、格式化日志，提升吞吐量
- 可靠传输
  - 通过共享内存 + 消息队列实现进程间高速、低锁通信
- 流量控制
  - 使用令牌桶算法防止下游存储被突发日志压垮
- 高可用
  - 守护进程运行、断点续传、子进程保活、信号平滑重启
- 模块化解耦
  - 采集、处理、输出三大模块可分离部署，独立扩展

2. 模块划分与通信

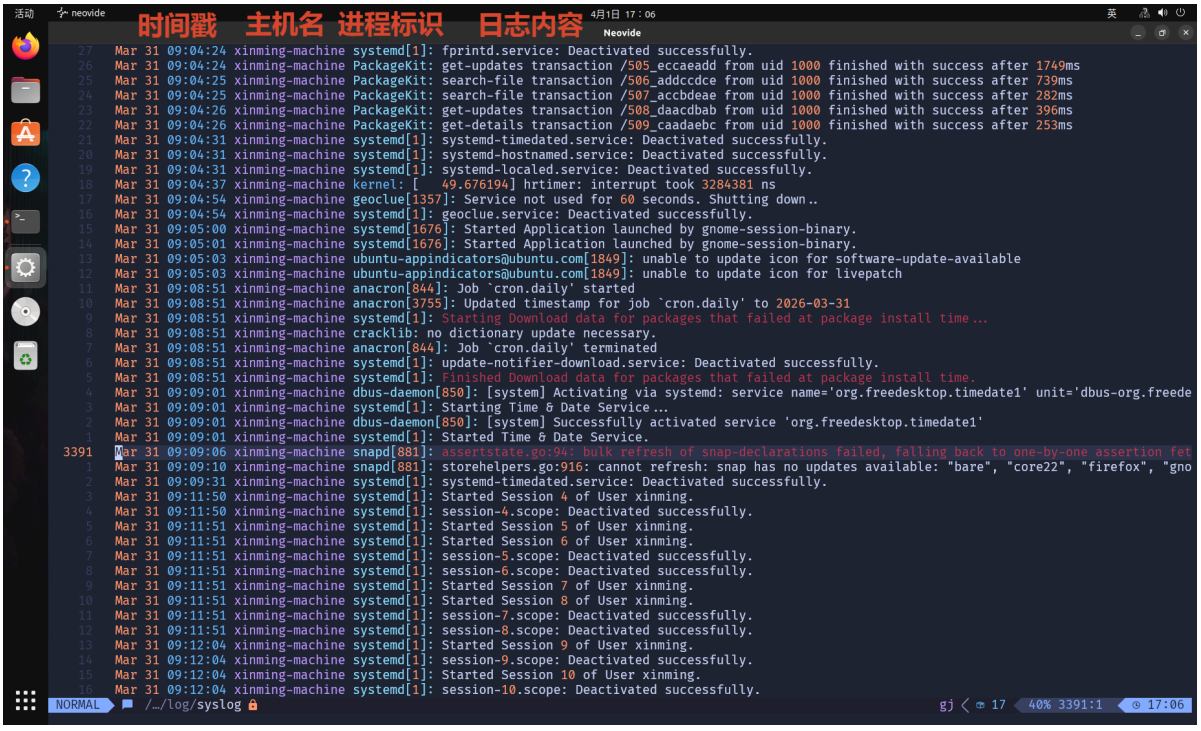
- `Collector`（采集）
  - 监控文件变化，读取新增日志，写入共享内存环形缓冲区（`ipc`）
- `Processor`（处理）
  - 多进程从共享内存消费日志，解析、结构化后发送至 `XSI` 消息队列

- `Outputer`（输出）
  - 守护进程从消息队列消费数据，经令牌桶限流后写入文件或远端

3. 理解认知

`/var/log/syslog` 系统日志的内容

```
xinming@log$ pwd
/var/log
xinming@log$ ls -l syslog
-rw-r----- 1 syslog adm 959253  4月  1 16:45 syslog
xinming@log$
```



4. 相关函数 openlog(3)

```
/*
    功能: 与系统日志系统建立连接
    参数:
        ident 标识信息, 通常为程序名或进程名
        option 控制选项 (如 LOG_CONS, LOG_PID 等)
        facility 日志来源类型 (如 LOG_DAEMON, LOG_USER 等)
```

```
*/
void openlog(const char *ident, int option, int facility);
/*
option 控制选项
    LOG_CONS 如果无法发送到 syslogd, 则直接输出到控制台
    LOG_PID 每条日志消息中包含当前进程的 PID
    LOG_NDELAY 立即打开连接 (默认是首次调用 syslog 时打开)
    LOG_NOWAIT 不等待可能创建的子进程 (已过时)
    LOG_PERROR 同时将日志输出到 stderr (Linux 特有)

facility 日志来源类型
    LOG_USER 用户级程序 (默认)
    LOG_DAEMON 系统守护进程
    LOG_AUTH 安全 / 认证消息
    LOG_LOCAL0 ~ LOG_LOCAL7 本地使用 (自定义)
    LOG_MAIL 邮件系统
    LOG_CRON cron 守护进程
    LOG_KERN 内核消息
*/

/*
    功能: 记录一条日志信息到系统日志
    参数:
        priority 优先级 (priority = facility | level 参数实际上需要 facility + level)
        format 格式化字符串
        ... 可变参数
*/
void syslog(int priority, const char *format, ...);
// 如果只传 level, 系统会自动使用 openlog() 中设置的 facility
syslog(LOG_CRIT, "严重错误, 需要人工介入");
// 明确指定了 facility 和 level, 会覆盖 openlog() 中设置的默认 facility
syslog(LOG_LOCAL0 | LOG_CRIT, "严重错误, 需要人工介入");
/*
priority 优先级
    LOG_EMERG   0  系统不可用 (最高优先级)
    LOG_ALERT   1  需要立即处理
```

```
    LOG_CRIT     2   严重情况
    LOG_ERR      3   错误条件
    LOG_WARNING  4   警告
    LOG_NOTICE   5   普通但重要的通知
    LOG_INFO     6   信息性消息
    LOG_DEBUG    7   调试信息 (最低优先级)
*/

// 功能: 关闭与日志系统的连接
void closelog(void);
```

示例：

```
int main() {
    // 初始化日志系统
    openlog("myprogram", LOG_CONS | LOG_PID | LOG_NDELAY, LOG_LOCAL0);

    // 记录不同级别的日志
    syslog(LOG_INFO, "程序启动成功，PID: %d", getpid());
    syslog(LOG_WARNING, "磁盘使用率超过80%%");
    syslog(LOG_ERR, "数据库连接失败: %s", "connection timeout");
    syslog(LOG_DEBUG, "调试信息: 变量 x = %d", 100);

    // 使用完整的优先级 (facility + level)
    syslog(LOG_LOCAL0 | LOG_CRIT, "严重错误，需要人工介入");

    // 关闭连接
    closelog();

    return 0;
}
```

```
# 查看 syslog 文件
neovide /var/log/syslog
# 搜索 a.out 相关日志信息  注意 '.' 要转义
:a\.out
```

在守护进程中，可将 `perror(3)` 和 `fprintf(3)` 等错误输出替换为 `syslog(3)`，统一记录到系统日志中

5. 实现逻辑

```
[ syslog 文件 ]
    ↓ (新增日志行)
[ Collector ] → 读取新增 → 写入共享内存 → 信号量通知
    ↓
[ Processor ] → 从共享内存读 → 解析日志行 → 提取级别 → 过滤（只保留 WARN/ERROR） → 格式化为 JSON → 发送到消息队列
    ↓
[ Outputer ] → 从消息队列接收 → 令牌桶限流 → 写入最终文件 (/tmp/loghawk.log)
```

`JSON` 结构

```
{
  "timestamp": "2025-04-01T12:34:56+08:00",  // 时间戳
  "level": "ERROR",  // 日志级别
  "host": "server01",  // 主机名或 IP
  "service": "order-service",  // 服务名
  "message": "database connection timeout",  // 日志内容，消息内容
  "extra": {  // 附加字段（自定义）
    "pid": 12345,  // 进程 ID
  }
}
```

```c
char *parse_syslog_to_json(const char *line)  // 将解析后的日志字段拼接成一个 JSON 字符串
{
    // 先通过解析从 line 中提取出 timestamp, level, host, service, message

    // 然后封装成 json
    char json[1024];
    // 格式化内容写入字符串
    snprintf(json, sizeof(json),
             "{\"timestamp\":\"%s\",\"level\":\"%s\",\"host\":\"%s\","
             "\"service\":\"%s\",\"message\":\"%s\"}",
             timestamp, level, host, service, message);
    return strdup(json);  // 系统调用, 分配内存给 json
}
```

6. 项目文件结构

```
loghawk/
├── ipc.h / ipc.c           # 共享内存 + 信号量封装
├── msgq.h / msgq.c         # 消息队列封装
├── token_bucket.h / token_bucket.c  # 令牌桶
├── collector.c             # 采集模块
├── processor.c             # 处理模块
├── outputer.c              # 输出模块
├── main.c                  # 主进程（启动/管理）
├── Makefile
```

`ipc.h` (共享内存 + 信号量)

```c
#ifndef IPC_H
#define IPC_H

```

```c
#define SHM_SIZE (4 * 1024 * 1024)  // 4MB
#define SEM_KEY 0x1234
#define SHM_KEY 0x5678

// 环形缓冲区结构
struct shm_buffer {
    int write_pos;
    int read_pos;
    char data[SHM_SIZE];
};

// 信号量操作封装
int sem_create(int key);
void sem_wait(int semid);
void sem_signal(int semid);
void sem_destroy(int semid);

// 共享内存封装
int shm_create(int key, size_t size);
void *shm_attach(int shmid);
void shm_detach(void *addr);
void shm_destroy(int shmid);

#endif
```

`msgq.h` 消息队列封装

```c
#ifndef MSGQ_H
#define MSGQ_H

#define MSG_MAX 1024
#define MSG_KEY 0x9ABC

struct msgbuf {
    long mtype;          // 消息类型（1：日志数据）
    char mtext[MSG_MAX];
};

```

```c
int msgq_create(int key);
void msgq_send(int msqid, const char *text, long type);
int msgq_recv(int msqid, char *buf, long type);
void msgq_destroy(int msqid);

#endif
```

`token_bucket.h` 令牌桶

```c
#ifndef TOKEN_BUCKET_H
#define TOKEN_BUCKET_H

struct token_bucket {
    int tokens;      // 当前令牌数
    int rate;        // 每秒生成令牌数
    int capacity;    // 桶容量
};

void token_bucket_init(struct token_bucket *tb, int rate, int capacity);
void token_bucket_add_tokens(struct token_bucket *tb, int count);
int token_bucket_get(struct token_bucket *tb, int need);

#endif
```

日志采集模块（`Collector`）

监控日志文件，轮询检测新增数据，写入共享内存

```c
#include "ipc.h"
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <signal.h>

```

```c
extern int shmid, semid;
extern struct shm_buffer *shm;

// 记录每个文件的偏移量（仅演示一个文件）
static char filename[256] = "/tmp/test.log";
static long offset = 0;
static ino_t last_inode = 0;

// 持久化偏移量（简单写文件）
void save_offset() {
    FILE *fp = fopen("/tmp/collector.offset", "w");
    if (fp) {
        fprintf(fp, "%ld\n", offset);
        fclose(fp);
    }
}

void load_offset() {
    FILE *fp = fopen("/tmp/collector.offset", "r");
    if (fp) {
        fscanf(fp, "%ld", &offset);
        fclose(fp);
    }
}

// 检测文件变化并读取
void monitor_file() {
    struct stat st;
    if (stat(filename, &st) == -1) {
        perror("stat");
        return;
    }

    // 处理日志轮转：inode 变化
    if (last_inode != 0 && last_inode != st.st_ino) {
        // 文件被轮转，重新打开
        offset = 0;
    }
    last_inode = st.st_ino;

```

```
    if (st.st_size > offset) {
        int fd = open(filename, O_RDONLY);
        if (fd < 0) {
            perror("open");
            return;
        }

        // 定位到上次读取位置
        lseek(fd, offset, SEEK_SET);
        char buf[4096];
        ssize_t n = read(fd, buf, sizeof(buf));
        if (n > 0) {
            // 写入共享内存
            // 注意：环形缓冲区写入需考虑跨边界情况，这里简化，只写单条
            // 实际应分段写入
            int write_pos = shm->write_pos;
            int remain = SHM_SIZE - write_pos;
            if (n < remain) {
                memcpy(shm->data + write_pos, buf, n);
                shm->write_pos = (write_pos + n) % SHM_SIZE;
            } else {
                // 分两段写
                memcpy(shm->data + write_pos, buf, remain);
                memcpy(shm->data, buf + remain, n - remain);
                shm->write_pos = n - remain;
            }
            // 发送信号量通知消费者
            sem_signal(semid);
        }
        offset += n;
        save_offset();
        close(fd);
    }
}

void collector_loop() {
```

```
    while (1) {
        monitor_file();
        usleep(100000);  // 100ms 轮询
    }
}

int main() {
    // 连接共享内存和信号量（由主进程创建）
    shmid = shm_get(SHM_KEY);
    shm = (struct shm_buffer *)shm_attach(shmid);
    semid = sem_get(SEM_KEY);  // 获取已有信号量

    load_offset();
    collector_loop();
    return 0;
}
```

日志处理模块（`Processor`）

多进程消费共享内存，解析日志，发送消息队列

```
#include "ipc.h"
#include "msgq.h"
#include <sys/wait.h>
#include <signal.h>
#include <unistd.h>

#define NUM_WORKERS 4  // 进程数

extern int shmid, semid;
extern struct shm_buffer *shm;
extern int msqid;

// 子进程工作函数
void worker(int id) {
    while (1) {
        // 等待信号量（有数据可读）
        sem_wait(semid);

```

```c
        // 从共享内存读取一条日志（简化：假设一行以换行符结束）
        int read_pos = shm->read_pos;
        // 实际应解析环形缓冲区中的完整行，这里取一条固定长度
        char line[512];
        int i = 0;
        while (shm->data[read_pos] != '\n' && i < 511) {
            line[i++] = shm->data[read_pos];
            read_pos = (read_pos + 1) % SHM_SIZE;
        }
        line[i] = '\0';
        // 更新读指针
        shm->read_pos = (read_pos + 1) % SHM_SIZE; // 跳过换行符

        // 模拟解析：添加时间戳和 JSON 格式
        char processed[1024];
        snprintf(processed, sizeof(processed),
                 "{\"timestamp\":\"%ld\", \"worker\":%d, \"msg\":\"%s\"}",
                 time(NULL), id, line);

        // 发送到消息队列
        msgq_send(msqid, processed, 1);
    }
}

int main() {
    // 连接资源
    shmid = shm_get(SHM_KEY);
    shm = (struct shm_buffer *)shm_attach(shmid);
    semid = sem_get(SEM_KEY);
    msqid = msgq_get(MSG_KEY);

    // 创建子进程池
    for (int i = 0; i < NUM_WORKERS; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            worker(i);
            exit(0);
```

```
        } else if (pid < 0) {
            perror("fork");
        }
    }

    // 父进程等待子进程退出（简单回收）
    for (int i = 0; i < NUM_WORKERS; i++) {
        wait(NULL);
    }
    return 0;
}
```

日志输出模块（`Outputer`）

守护进程 + 令牌桶限流，从消息队列接收并写入文件

```
#include "msgq.h"
#include "token_bucket.h"
#include <sys/time.h>
#include <signal.h>
#include <fcntl.h>
#include <unistd.h>

static struct token_bucket tb;
static int msqid;
static int output_fd;

void sig_alrm(int signo) {
    // 每秒添加令牌
    token_bucket_add_tokens(&tb, tb.rate);
    alarm(1);  // 重新设置定时器
}

void daemonize() {
    pid_t pid = fork();
    if (pid < 0) exit(1);
    if (pid > 0) exit(0);  // 父进程退出
    setsid();
    chdir("/");
```

```
    umask(0);
    close(STDIN_FILENO);
    close(STDOUT_FILENO);
    close(STDERR_FILENO);
    open("/dev/null", O_RDWR);  // 重定向标准输入
    dup(0); dup(0);              // 标准输出、错误
}

int main() {
    // 守护进程化
    daemonize();

    // 初始化令牌桶：速率 1000 条/秒，容量 2000
    token_bucket_init(&tb, 1000, 2000);

    // 设置定时器每秒生成令牌
    signal(SIGALRM, sig_alrm);
    alarm(1);

    // 打开输出文件
    output_fd = open("/var/log/loghawk.log", O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (output_fd < 0) {
        // 回退到 /tmp
        output_fd = open("/tmp/loghawk.log", O_WRONLY | O_CREAT | O_APPEND, 0644);
    }

    // 连接消息队列
    msqid = msgq_get(MSG_KEY);

    char buf[MSG_MAX];
    while (1) {
        if (msgq_recv(msqid, buf, 1) == 0) {
            // 限流检查
            if (token_bucket_get(&tb, 1)) {
                write(output_fd, buf, strlen(buf));
                write(output_fd, "\n", 1);
            } else {
                // 令牌不足，可丢弃或延迟，这里简单丢弃并记录
```

```
                static int dropped = 0;
                dropped++;
                if (dropped % 100 == 0) {
                    // 记录丢弃统计
                }
            }
        }
    }
    close(output_fd);
    return 0;
}
```

主进程（启动与资源管理）

负责创建 `IPC` 资源，启动 `Collector、Processor、Outputer`，处理信号实现退出

```
#include "ipc.h"
#include "msgq.h"
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

static int shmid, semid, msqid;
static void *shm_addr;
static pid_t collector_pid, processor_pid, outputer_pid;

void cleanup() {
    // 终止子进程
    if (collector_pid) kill(collector_pid, SIGTERM);
    if (processor_pid) kill(processor_pid, SIGTERM);
    if (outputer_pid) kill(outputer_pid, SIGTERM);
    // 等待子进程退出
    if (collector_pid) waitpid(collector_pid, NULL, 0);
    if (processor_pid) waitpid(processor_pid, NULL, 0);
    if (outputer_pid) waitpid(outputer_pid, NULL, 0);
    // 销毁 IPC 资源
    shm_detach(shm_addr);
```

```
    shm_destroy(shmid);
    sem_destroy(semid);
    msgq_destroy(msqid);
    exit(0);
}

void signal_handler(int sig) {
    if (sig == SIGTERM || sig == SIGINT) {
        cleanup();
    }
}

int main() {
    // 设置信号处理
    signal(SIGTERM, signal_handler);
    signal(SIGINT, signal_handler);

    // 创建 IPC 资源
    shmid = shm_create(SHM_KEY, sizeof(struct shm_buffer));
    shm_addr = shm_attach(shmid);
    semid = sem_create(SEM_KEY);
    msqid = msgq_create(MSG_KEY);

    // 启动 Collector
    collector_pid = fork();
    if (collector_pid == 0) {
        execl("./collector", "collector", NULL);
        perror("execl collector");
        exit(1);
    }

    // 启动 Processor
    processor_pid = fork();
    if (processor_pid == 0) {
        execl("./processor", "processor", NULL);
        perror("execl processor");
        exit(1);
    }

```

```
    // 启动 Outputer
    outputer_pid = fork();
    if (outputer_pid == 0) {
        execl("./outputer", "outputer", NULL);
        perror("execl outputer");
        exit(1);
    }

    // 主进程等待子进程退出
    int status;
    waitpid(collector_pid, &status, 0);
    waitpid(processor_pid, &status, 0);
    waitpid(outputer_pid, &status, 0);

    cleanup();  // 正常情况下不会执行到这里
    return 0;
}
```